Informe - Torneo de Komikan Concurrente

Tarea 2 Sistemas Operativos

Integrantes: GianFranco Caleni Gabriel Guaiquipán

C con pthreads y semáforos POSIX, compilado con `gcc -Wall -Wextra -std=c17`

1 Diseño del esquema de sincronización

Arquitectura. Todo corre en un solo proceso con hilos. Hay un hilo Jugador por cliente (N), un hilo Matchmaker, y por cada partida un hilo Juez más las fichas: el jugador Puma ejecuta el ciclo del Puma y el jugador Perros lanza 12 hilos Perro (13 fichas concurrentes por mesa). El Matchmaker empareja jugadores en orden de llegada (el primero es Puma, el segundo Perros) y, antes de abrir una mesa, hace `sem_wait` sobre un semáforo POSIX con valor inicial M: si las M mesas están ocupadas queda bloqueado en el kernel (espera pasiva, sin busy-waiting) y los jugadores emparejados duermen en su propia variable de condición. El Juez hace `sem_post` al cerrar la mesa.

Exclusión mutua en el tablero: Juez como único escritor. Ninguna ficha modifica el tablero. Cada ficha (1) decide su movimiento leyendo el tablero bajo el mutex de la mesa, (2) espera su retardo aleatorio (50-300 ms, semilla reproducible), y (3) envía una solicitud (origen, destino) a una cola FIFO de la mesa y se bloquea en su propia variable de condición. El Juez toma las solicitudes en orden de llegada, las valida (adyacencia, casilla libre, salto válido sobre un perro, ficha viva y en el origen declarado) y las aplica de forma atómica. Así "la primera ficha que reclama una casilla libre se la queda": si dos fichas piden la misma casilla, la segunda solicitud se procesa ya con la casilla ocupada y recibe RECHAZADO. Como decidir y actuar están separados por el retardo, las decisiones pueden quedar obsoletas y el Juez es quien garantiza la consistencia (verificada además con invariantes al cerrar cada mesa).

Primitiva	**Protege/función**
`sem_t` nombrado (`sem_open`, valor M)	Límite de mesas activas. Se hace `sem_unlink` inmediatamente tras abrirlo: si el proceso muere no queda nada huérfano. Nombrado porque macOS no implementa `sem_init`.
mutex por mesa + cond del Juez + cond por ficha	Tablero, cola FIFO, contadores y estado de la partida. Una cond por ficha evita despertar a las 13 (sin thundering herd). Esperas siempre en bucle `while` (predicado).
mutex + cond de la cola de espera y por jugador	Llegada de jugadores al Matchmaker y asignación de mesa a cada jugador.
mutex de log y de estadísticas	Salida atómica de tableros multilínea; contadores globales (mesas activas, máximo observado, resultados).

Resiliencia. Desconexión: cada jugador/ficha registra con `pthread_cleanup_push` un handler que, si el hilo muere de forma abrupta (`pthread_exit`, incluso con el mutex de la mesa tomado), primero libera el mutex y luego avisa al Juez; éste declara ganador al oponente por abandono, despierta a todos, y destruye mutex/conds de la mesa antes de liberar el semáforo. SIGINT/SIGTERM: el manejador solo asigna un `atomic_int` y hace `sem_post` (ambos async-signal-safe) para despertar al Matchmaker; los demás hilos usan esperas con tiempo límite (≤ 100 ms) y revisan la bandera, por lo que cierran las partidas, cancelan a los jugadores en cola, hacen join de todos los hilos y liberan mutex, conds, semáforo y memoria (0 leaks verificado con ASan/LeakSanitizer).

2 Análisis de riesgos de concurrencia

- **Deadlock.** Las 13 fichas de una mesa compiten por un solo mutex y ninguna lo mantiene mientras espera otro recurso, así que no hay espera circular. Jerarquía global de locks: mutex de mesa $\rightarrow$ mutex de log (nunca al revés); el Matchmaker bloquea en el semáforo sin sostener ningún mutex; el Juez asigna jugadores antes de tomar su mutex y actualiza estadísticas después de soltarlo. Una ficha esperando respuesta no bloquea al Juez, que siempre responde (también con `MV_OVER` al terminar). Un hilo caído con el mutex tomado no deja la mesa colgada gracias a los cleanup handlers.

- **Livelock.** Dos fichas que reintentan sin parar la misma casilla se evitan porque cada ciclo rehace la decisión con el tablero actual, tras un retardo aleatorio independiente por ficha (RNG con semilla propia), y un rechazo solo cuesta un ciclo. Además hay un límite de movimientos (`MAX_MOVES`, empate) que garantiza que toda partida termine aunque los perros se queden cautelosos.

- **Starvation.** Las solicitudes se atienden en orden FIFO de llegada y la cola de emparejamiento es FIFO, por lo que ningún jugador queda postergado indefinidamente mientras haya mesas. El mutex de pthread no es equitativo, pero las secciones críticas son $\mathcal{O}(1)$ y los hilos duermen 50-300 ms entre accesos (contención baja). Si N es impar, el último jugador queda ”sin pareja” y se informa (no espera para siempre).

- **Espera pasiva.** Los pares que no consiguen mesa duermen bloqueados (`sem_wait` en el Matchmaker, `cond_wait` en cada jugador): no hay bucles de sondeo. Las únicas esperas con tiempo límite (100 ms) son las que deben vigilar la bandera de SIGINT, ya que un manejador de señal no puede despertar variables de condición de forma segura. Medido con $N = 20$ y $M = 2$ (cola saturada durante toda la ejecución): 0,8% de uso de CPU.

- **Otros.** Lost wakeup: todas las esperas son `while` (predicado) con el estado bajo el mismo mutex. Use-after-free: el Juez destruye la mesa solo cuando `participants == 0` (todos los hilos de la mesa salieron y los Perros fueron join-eados).

3 Evaluación de las decisiones de diseño

Alternativas. (a) Mutex por casilla con adquisición en orden ascendente de índice: máximo paralelismo, pero un movimiento toca 2-3 casillas (salto: origen, intermedia, destino), el log del tablero exige tomar las 28 y los errores de orden causan deadlocks difíciles de depurar. (b) Un mutex grueso por mesa con cada ficha mutando directamente: simple, pero la validación de reglas queda dispersa en 13 hilos y no existe el ”Juez” como autoridad. (c) Juez único escritor + cola FIFO (elegida): la validación de reglas, la detección de victoria y el log están en un solo lugar, es fácil de razonar y de probar. El costo es un cambio de contexto por movimiento y que el Juez serializa los movimientos de una mesa; esto es irrelevante porque los movimientos son $\mathcal{O}(1)$, cada ficha duerme 50-300 ms y el paralelismo real se da entre mesas (hasta M partidas simultáneas, cada una con su propio mutex).

Resultados medidos (Linux, 1 núcleo): en el stress test, 13 hilos con retardo de 1 ms emiten 500 intentos en $\sim$53 ms (=9.400 solicitudes/s), 223 rechazados correctamente y tablero consistente; el máximo de mesas simultáneas observado nunca superó M; la suite de 12 pruebas pasa en 12/12 sin avisos de ThreadSanitizer, ni errores de Address Sanitizer/UBSan, ni leaks. Supuestos (figura del enunciado): adyacencia de 8 vecinos en la grilla y trono de 3 nodos (TOP-(2,4), TOP-LEFT, TOP-RIGHT, LEFT-(1,4), RIGHT-(3,4)); ambos configurables en `build_tables()`.